Penjelasan kode program (main.py)

1. linked_list.py

File ini berisi class dasar node yang digunakan oleh Queue dan Stack.

```
from typing import Optional


class NodeLL:
    def __init__(self, data: str):
        self.data = data
        self.next: Optional['NodeLL'] = None


class EdgeNode:
    def __init__(self, dest: str, bobot: int):
        self.dest = dest
        self.bobot = bobot
        self.next: Optional['EdgeNode'] = None
```

Penjelasan

NodeLL berfungsi sebagai Node dasar untuk Linked List.

Atributnya terdiri dari :

- Data untuk menyimpan data bertipe string.
- Next unt ukmenunjuk ke node berikutnya.

Fungsinya digunakan sebagai elemen dasar untuk membangun Queue dan Stack.

EdgeNode berfungsi sebagai Node khusus untuk menyimpan data graf.

Atributnya terdiri

- Dest untuknama tujuan/simpul tujuan.
- Bobot untuk nilai bobot (weight).
- Next untuk menunjuk ke edge berikutnya.

Fungsinya digunakan pada representasi graf dengan adjacency list.

---

2. queue_ll.py

Implementasi Queue menggunakan Linked List.

```python
from typing import Optional
from linked_list import NodeLL


class QueueLL:
    def __init__(self):
        self.head: Optional[NodeLL] = None
        self.tail: Optional[NodeLL] = None

    def enqueue(self, data: str):
        new_node = NodeLL(data)

        if not self.tail:
            self.head = self.tail = new_node
            return
        self.tail.next = new_node
        self.tail = new_node

    def dequeue(self) -> Optional[str]:
        if not self.head:
            return None

        temp = self.head
        self.head = self.head.next

        if not self.head:
            self.tail = None

        return temp.data

    def is_empty(self) -> bool:
        return self.head is None
```

Penjelasannya

Queue menggunakan prinsip FIFO (First In First Out).

Atribut terdiri dari :

- head: elemen paling depan.
- tail: elemen paling belakang.

Method yang digunakan :

- enqueue(data) untuk menambahkan data ke belakang queue.

- dequeue() untuk menghapus dan mengembalikan data dari depan queue.

is_empty() untuk mengembalikan True jika queue kosong.

3. stack_ll.py

Implementasi Stack menggunakan Linked List.

```
from typing import Optional
from linked_list import NodeLL


class StackLL:
    def __init__(self):
        self.top: Optional[NodeLL] = None

    def push(self, data: str):
        new_node = NodeLL(data)
        new_node.next = self.top
        self.top = new_node

    def pop(self) -> Optional[str]:
        if not self.top:
            return None
```

```
    temp = self.top
    self.top = self.top.next
    return temp.data

def is_empty(self) -> bool:
    return self.top is None
```

Penjelasannya

Stack menggunakan prinsip LIFO (Last In First Out).

Atribut menggunakan top: elemen paling atas.

Methodenya terdiri dari:

- push(data) untuk menambahkan data ke atas stack.
- pop() untuk menghapus dan mengembalikan data dari atas stack.
- is_empty() untuk mengembalikan True jika stack kosong.